COMPARISON

React to PDF: four ways compared

Updated 15 Dec 2026

KEY TAKEAWAYS

- Four ways: build with react-pdf's own components, use the browser's print dialog, capture the DOM to an image, or print the page in headless Chromium.
- react-pdf builds the PDF from its own primitives and style objects; your existing components and stylesheets don't carry over, and Tailwind works only through its `@react-pdf/tailwind` subset.
- html2pdf.js renders content into an image, so its text is not selectable or searchable, per its README.
- The print dialog and headless Chromium print your real page with its CSS, Tailwind included, and take page breaks from CSS `break-*` rules.
- papyrus is a young option inside the fourth way; react-pdf is the project built for making a PDF from scratch.

CONTENTS

1 Four React to PDF approaches at a glance 3
2 react-pdf: build the PDF from components 3
3 The browser’s print dialog 5
4 Capture the DOM: html2canvas with jsPDF, or html2pdf.js 5
5 Headless Chromium: Playwright, Puppeteer, papyrus 7
6 What usually decides it 7
7 Where papyrus fits 8
8 What to remember 8

There are four ways to go from React to PDF. You can (1) build the PDF from react-pdf's own components, (2) use the browser's print dialog on your React page, (3) capture the DOM as an image and wrap it in a PDF with html2canvas and jsPDF, or html2pdf.js, or (4) render your React page in headless Chromium and print it with Playwright or Puppeteer. A rough rule: pick 1 when the PDF is a document of its own, 2 when readers print what they see, 3 for a quick download button in the browser, and 4 when the PDF must match your page and can be made on a server or in CI.

Below, each approach in turn, then the three questions that usually settle it: page breaks, Tailwind and tables.

1. Four React to PDF approaches at a glance

	react-pdf	Print dialog	html2canvas + jsPDF, html2pdf.js	Headless Chromium
Where it runs	Browser or Node	The reader's browser	Browser only	Node on a server or in CI
Your components and CSS	Rewritten with its primitives and style objects	Used as they are	Read from the DOM; not every CSS property is supported	Used as they are
Tailwind classes	A subset, through `@react-pdf/tailwind`	Yes	Whatever CSS html2canvas supports	Yes
Page breaks	`break`, `wrap`, `minPresenceAhead` props	CSS `break-*`	html2pdf.js modes, part of CSS	CSS `break-*`
Text in the PDF	Built by react-pdf	Printed by the browser	An image: not selectable (html2pdf.js)	Printed by the browser
Fonts	`Font.register`, TTF and WOFF	The page's own fonts	Drawn into the image	The page's fonts; wait for them to load
Typical use	A document designed for PDF	"Print this page"	A small download button	Reports and invoices that must match the page

2. react-pdf: build the PDF from components

react-pdf's README calls it a React renderer for creating PDF files in the browser and on the server. You write `Document`, `Page`, `View` and `Text`, style them with `StyleSheet.create` or a `style` prop, and it lays out the PDF. Its component list also has `PDFViewer` to show the PDF in the page and `PDFDownloadLink`, a web-only link that generates the PDF and downloads it. In Node, the README writes to a file with `ReactPDF.render`.

```
import { Document, Page, Text, View } from "@react-pdf/renderer";

export const Report = ({ sections }) => (
  <Document>
    <Page size="A4" style={{ padding: 40 }}>
      <Text fixed render={({ pageNumber, totalPages }) => `${pageNumber} /
${totalPages}`} />
      {sections.map((s, i) => (
        <View key={s.title} break={i > 0}>
          <Text minPresenceAhead={40}>{s.title}</Text>
          <Text>{s.body}</Text>
        </View>
      ))}
    </Page>
  </Document>
);
```

That's a document with a page number on every page, where each section starts a new page; three sections gave us three pages. `Page` is A4 by default and wraps content onto new pages by default. The `break` prop starts a new page. `fixed` repeats an element on every page, and `render` receives `pageNumber` and `totalPages`. `minPresenceAhead` is a hint that no page break should fall between the element and the siblings that follow it within that many points. For text there are also `orphans` and `widows`, both defaulting to 2.

The docs carry one warning: `wrap={false}` only works for content that fits on one page. Taller content gets squeezed into the space left, overlaps and can disappear. Keep it for a card or a table row.

react-pdf includes the standard PDF fonts Courier, Helvetica and Times-Roman. For anything else, `Font.register` takes a family and a `src`, and the docs say only TTF and WOFF files are supported.

Where it fits: a document that exists only as a PDF, made in the browser or on a server, with fine control over pages. **Where it doesn't:** reusing the components and stylesheets you already have. react-pdf follows the React primitives approach, so the page you show on screen isn't the page it prints, and you maintain both.

3. The browser’s print dialog

`window.print()` opens the print dialog, and MDN notes that it blocks while the dialog is open. The dialog prints to paper or saves a PDF. You style the result in an `@media print` block, which MDN says applies only to printed content:

```
export const PrintButton = () => (
  <button className="no-print" onClick={() => window.print()}>Print</button>
);
```

```
@media print { .no-print { display: none; } }
```

The sheet size comes from `@page`, covered in css page size a4. Breaks come from `break-before`, `break-after` and `break-inside`; MDN lists `break-before: page` from Chrome 50, Edge 79, Firefox 65 and Safari 10. The page-break article has the details and the cases where the rules don’t apply.

Where it fits: readers print what they see, and you need no server. **Where it doesn’t:** making files in a build step or on a server. The reader also controls the dialog, paper and settings included.

---

4. Capture the DOM: html2canvas with jsPDF, or html2pdf.js

html2canvas reads the DOM and the styles applied to its elements and draws them onto a canvas. Its README says it doesn’t take a real screenshot but builds one from what’s on the page, so the result may not match what the browser shows. Each CSS property has to be implemented by hand, it says, so some aren’t supported yet. The README, as we read it on 11 November 2026, still describes the script as being in a “very experimental state” and advises against using it in production. jsPDF is the half that writes the PDF; its README says its `html` method depends on html2canvas. html2pdf.js wraps the two together.

```
import { useRef } from "react";
import html2pdf from "html2pdf.js";

export function InvoiceDownload({ children }) {
  const ref = useRef(null);
  const download = () =>
    html2pdf()
      .set({ margin: 10, filename: "invoice.pdf", jsPDF: { unit: "mm", format:
"a4" } })
      .from(ref.current)
      .save();
  return (
    <>
      <div ref={ref}>{children}</div>
      <button onClick={download}>Download PDF</button>
    </>
  );
}
```

The margin is in jsPDF units, so 10 means 10 mm here. By default html2pdf.js respects most CSS `break-before`, `break-after` and `break-inside` rules, and it has three page-break modes: `css`, `legacy` and `avoid-all`. The README adds that the `css` mode recognizes only `always`, `left` and `right` for breaks before and after, and `avoid` inside. `page` isn't on that list, so test your breaks in the downloaded file.

The README is plain about the limits. It renders all content into an image and puts the image in a PDF, so text isn't selectable or searchable and files get large. It won't run in Node. Content larger than a canvas's maximum size fails to render.

Where it fits: a download button in the browser for a short, simple page, when an image-based PDF is acceptable. **Where it doesn't:** documents people copy or search, long documents, and anything on a server.

5. Headless Chromium: Playwright, Puppeteer, papyrus

Render your React page in a headless browser and print it. Playwright's `page.pdf()` and Puppeteer's `Page.pdf()` both print with the `print` CSS media type, so your page, CSS and fonts are what gets printed, with no canvas step in between. It runs where you can launch a browser: a server, a CI job or your own machine. Playwright generate PDF from HTML walks through a script, the options and a page-count check.

Where it fits: reports, invoices and anything that must match the page and be produced without a person clicking Print. **Where it doesn't:** a browser-only app with no backend, or anywhere you can't run Chromium.

---

6. What usually decides it

Page breaks. If you want to control breaks from props, react-pdf has the most direct tools: `break`, `wrap`, `fixed` and `minPresenceAhead`. If your page is already HTML, the approaches that print it (2 and 4) break with plain CSS, so `tr { break-inside: avoid }` keeps a table row whole, and the page-break article lists what browsers honour. html2pdf.js offers its `pagebreak` option and the subset of CSS values above.

Tailwind. Tailwind's docs say it generates the CSS from the classes you actually use. A printed page applies that CSS as it is, so approaches 2 and 4 keep your Tailwind styling, and the `print:` variant, which applies styles only when printing, hides buttons and navigation. html2canvas rebuilds each CSS property itself, so check its output. react-pdf takes style objects, but its `@react-pdf/tailwind` package has a `tw()` function that turns a compatible subset of Tailwind classes into those objects, skipping unknown classes with a warning. You still rewrite the markup as react-pdf components and pass `tw("…")` to `style`.

Tables. react-pdf's component list has no table, so a table is rows of `View` and `Text`. In the other three, an HTML `table` stays a table.

7. Where papyrus fits

papyrus, a toolkit by n-uf for documents rendered as HTML and exported as PDF, sits inside approach 4. `@n-uf/papyrus-react` provides React components for pages and content blocks, such as `PDFPage`; `printStyles`, which its README describes as A4 sheet CSS plus print fixes for those blocks; and a Tailwind v4 stylesheet, `@import "@n-uf/papyrus-react/tailwind.css"`. The engine, `@n-uf/papyrus`, prints the pages with headless Chromium through Playwright and checks that the PDF's page count matches the page elements and the expected `pageCount`, so a page that overflowed is never saved. Its CLI needs the server that renders your documents to be running first, or `file:` URLs for plain HTML.

The React package ships TypeScript source, so a Next.js app lists it in `transpilePackages`:

```
// next.config.mjs
export default { transpilePackages: ["@n-uf/papyrus-react"] };
```

papyrus is young. Its versions are dates (`YY.M.R`), not semantic versions, so any release may break things. Pin an exact version and read the CHANGELOG before you upgrade. It supports React only, and it never draws a PDF itself: Chromium does the printing. To build a PDF from scratch with components made for it, react-pdf is the project for that. To print your own page with a page-count check, see html to pdf a4 with papyrus. papyrus and n-uf are not affiliated with Microsoft (Playwright), Google (Puppeteer, Chrome), the Chromium project, react-pdf, html2canvas, jsPDF, html2pdf.js or Tailwind Labs.

8. What to remember

- Choose by how much of your page has to survive: none of it (react-pdf), all of it (print dialog, headless Chromium), or a picture of it (html2pdf.js).
- html2pdf.js output is an image, so its text isn't selectable.
- Page breaks and Tailwind are where the four approaches differ most.

Questions people ask

What is the best React to PDF library?

There isn't one best choice; it depends on the job. react-pdf suits a document that exists only as a PDF, the print dialog suits readers printing what they see, html2pdf.js a quick client-side download where an image-based PDF is fine, and headless Chromium a PDF that must match your page, made on a server or in CI.

How do I download a React component as a PDF?

In the browser, pass the component's DOM element to `html2pdf().from(element).save()`, or build the document with react-pdf and offer it through its `PDFDownloadLink`, which react-pdf's docs describe as a web-only link that generates and downloads the PDF. The first captures how the page looks, as an image. The second builds the PDF from react-pdf's own components.

How do I control page breaks in a React PDF?

In react-pdf, use the `break` prop to start a new page, `wrap={false}` to keep a short block together, and `minPresenceAhead` to keep a heading with what follows. When you print a real page, use the CSS `break-before`, `break-after` and `break-inside` properties instead.

Can I use Tailwind classes in a React PDF?

Yes, fully, if the PDF is printed from your rendered page: Tailwind generates ordinary CSS from the classes in your markup, and the print dialog and headless Chromium apply it. react-pdf has a separate `@react-pdf/tailwind` package whose `tw()` function turns a compatible subset of Tailwind classes into react-pdf style objects. html2canvas redraws styles itself and doesn't support every CSS property.

How do I export a table to PDF in React?

On a printed page, use an HTML `table` and add `tr { break-inside: avoid }` so a row doesn't split. In react-pdf, build the table from `View` and `Text` rows, since its component list has no table, and use `wrap={false}` on a row; its docs suggest that for short blocks such as a table row.

Sources

1. react-pdf README (@react-pdf/renderer)↗ checked 11 Nov 2026
2. react-pdf docs: Components↗ checked 11 Nov 2026
3. react-pdf docs: PDFDownloadLink↗ checked 11 Nov 2026
4. react-pdf docs: Page↗ checked 11 Nov 2026
5. react-pdf docs: Page wrapping↗ checked 11 Nov 2026
6. react-pdf docs: Orphan and widow protection↗ checked 11 Nov 2026
7. react-pdf docs: Dynamic content↗ checked 11 Nov 2026
8. react-pdf docs: Fonts↗ checked 11 Nov 2026
9. react-pdf docs: Styling↗ checked 11 Nov 2026
10. react-pdf docs: Tailwind↗ checked 11 Nov 2026
11. MDN: window.print()↗ checked 11 Nov 2026
12. MDN: Printing↗ checked 11 Nov 2026
13. MDN: break-before↗ checked 11 Nov 2026
14. html2canvas README↗ checked 11 Nov 2026
15. jsPDF README↗ checked 11 Nov 2026
16. html2pdf.js README↗ checked 11 Nov 2026
17. Tailwind CSS: Styling with utility classes↗ checked 11 Nov 2026
18. Tailwind CSS: Hover, focus, and other states↗ checked 11 Nov 2026
19. Playwright: Page, page.pdf()↗ checked 11 Nov 2026
20. Puppeteer: Page.pdf()↗ checked 11 Nov 2026
21. Next.js: transpilePackages↗ checked 11 Nov 2026
22. papyrus README (@n-uf/papyrus v26.11.1)↗ checked 11 Nov 2026
23. papyrus CLI help (@n-uf/papyrus v26.11.1, src/pdf/cli.js)↗ checked 11 Nov 2026
24. papyrus-react README (@n-uf/papyrus-react v26.11.1)↗ checked 11 Nov 2026